

Project 14: Fatigue Prediction Beyond MLE

MUSULURU JEEVAN RAGHA SOURYA

MS25MTECH11002

Abstract

This project investigates whether a single best-fit fatigue law for additively manufactured Ti-6Al-4V is as reliable as it appears. Using S–N data from the FatigueData-AM2022 database (3,749 data points from 385 datasets, L-PBF route), three methods are applied to the same physics-based Basquin log–log model: Maximum Likelihood Estimation (MLE), Bayesian regression via PyMC, and Gaussian Process Regression (GPR) via scikit-learn. MLE produces a single confident-looking line. Bayesian inference reveals the distribution of plausible parameter values behind that line. GPR removes the straight-line constraint entirely and shows how uncertainty grows where data are sparse. Together, the three methods tell a more honest story than MLE alone: the apparent precision of the best-fit Basquin law is partly an illusion created by a point-estimate mindset in a highly scattered dataset. As an extension GPR with RBF kernel and polynomial Kernel were trained and tested by splitting the data based on data set ids and results were compared.

Introduction

Fatigue life prediction is a critical aspect of engineering design, particularly for materials such as additively manufactured Ti-6Al-4V, where variability in processing conditions leads to significant scatter in mechanical properties. Accurate estimation of the stress–life (S–N) relationship is essential for ensuring reliability and safety in structural applications. However, traditional approaches often rely on simplified models that may not fully capture the inherent uncertainty present in fatigue data.

In this study, the Basquin model is used as the underlying physics-based framework to describe the relationship between stress amplitude and fatigue life in log–log space. Three different modeling approaches are applied to the same dataset: Maximum Likelihood Estimation (MLE), Bayesian regression, and Gaussian Process Regression (GPR). MLE provides a single best-fit estimate, Bayesian regression extends this by quantifying uncertainty in model parameters, and GPR further generalizes the approach by learning the functional relationship directly from data without assuming a fixed form.

The objective of this analysis is to evaluate how each method represents both the central trend and the associated uncertainty in fatigue behavior. By comparing these approaches on the same dataset, the study aims to identify their strengths and limitations, particularly in the presence of significant data scatter and sparse regions. This enables a more informed understanding of which modeling strategy is most appropriate for fatigue prediction in real-world scenarios.

1. Dataset and Subset

1.1 Source

The FatigueData-AM2022 database contains fatigue data and metadata for additively manufactured alloys across S–N, ϵ –N, and da/dN– ΔK regimes. The full S–N sheet has 15,146 rows spanning 2,460 parameter entries.

1.2 Filtering

The following filters were applied sequentially to isolate a scientifically comparable subset:

- Material: Ti-6Al-4V (matched on “ti-6al-4v”, “ti64”)
- AM route: L-PBF (Laser Powder Bed Fusion) only
- Data quality: rows with valid $\text{life_N} > 0$ and $\text{stress_amplitude_MPa} > 0$

Final subset: 3,749 data points from 385 unique datasets.

1.3 Log–log transformation

The Basquin model is linear in log–log space. Both variables were log-transformed before any fitting:

$$x = \log_{10}N \quad \{\text{range: 2.19 to 9.48}\}$$

$$y = \log_{10}(\sigma_a) \quad \{\text{range: 0.76 to 3.04}\}$$

1.4 Data Splitting flow for Polynomial Kernel:

- The dataset consisting of **385 unique datasets** was divided into training and testing sets using an **80:20 ratio**.
- Splitting was performed by **randomly shuffling dataset IDs**, ensuring that all data points from a given dataset remain grouped together.
- This grouped splitting prevents data leakage and ensures independence between training and testing sets.
- Approximately **80% of the dataset IDs were assigned to the training set**, and the remaining **20% were assigned to the test set**.

- The splitting process was repeated for **two independent iterations**, each with a different random shuffle to assess robustness.
- For each iteration:
 - Training data was used to fit the models.
 - Testing data (unseen during training) was used to evaluate model performance.
 - The training dataset was **reset for each iteration**, ensuring independent evaluation.
- Two Gaussian Process Regression (GPR) models were trained on the same training data:
 - **RBF kernel**: captures smooth, local variations in the fatigue relationship.
 - **Polynomial kernel**: captures global trends similar to the Basquin model.
- In each iteration, both models were **trained from scratch**, and their **kernel hyperparameters were optimized independently** using marginal likelihood maximization.
- Model predictions were generated for both training and testing data, and performance was evaluated using **Root Mean Square Error (RMSE)**.
- This workflow enables a fair comparison between kernels and ensures that results are not dependent on a single data split, providing a reliable assessment of model generalization and predictive capability.

2. Maximum Likelihood Estimation (MLE)

Maximum Likelihood Estimation (MLE) with Basquin Model

Following the dataset preparation, the Basquin stress–life relationship is first estimated using a deterministic Maximum Likelihood Estimation (MLE) approach.

2.1 Framework

- The Basquin S–N law in its original form is:

$$\sigma_a = A \cdot N^B$$

- Taking logarithm (base 10) transforms the model into a linear form:

$$\log_{10}(\sigma_a) = \log_{10}(A) + B \cdot \log_{10}(N)$$

- The variables are defined as:

$$x = \log_{10}(N), \quad y = \log_{10}(\sigma_a)$$

- This formulation treats stress as the response variable and fatigue life as the predictor, consistent with the Basquin convention.

2.2 Design Matrix

The linear model is expressed in matrix form as:

$$\mathbf{y} = \mathbf{X}\boldsymbol{\beta}$$

where the design matrix is:

$$\mathbf{X} = \begin{bmatrix} 1 & \log_{10}(N_1) \\ 1 & \log_{10}(N_2) \\ \vdots & \end{bmatrix}$$

with shape:

$$(3749 \times 2)$$

- The first column corresponds to the intercept term $\log_{10}(A)$
- The second column contains the log-transformed fatigue life values

2.3 Parameter Estimation

Under the assumption of Gaussian noise, the MLE solution is equivalent to the Ordinary Least Squares (OLS) solution:

$$\boldsymbol{\beta} = (\mathbf{X}^T \mathbf{X})^{-1} \mathbf{X}^T \mathbf{y}$$

In practice, this is computed using a numerically stable least-squares solver based on Singular Value Decomposition (SVD), avoiding explicit matrix inversion.

2.4 Results

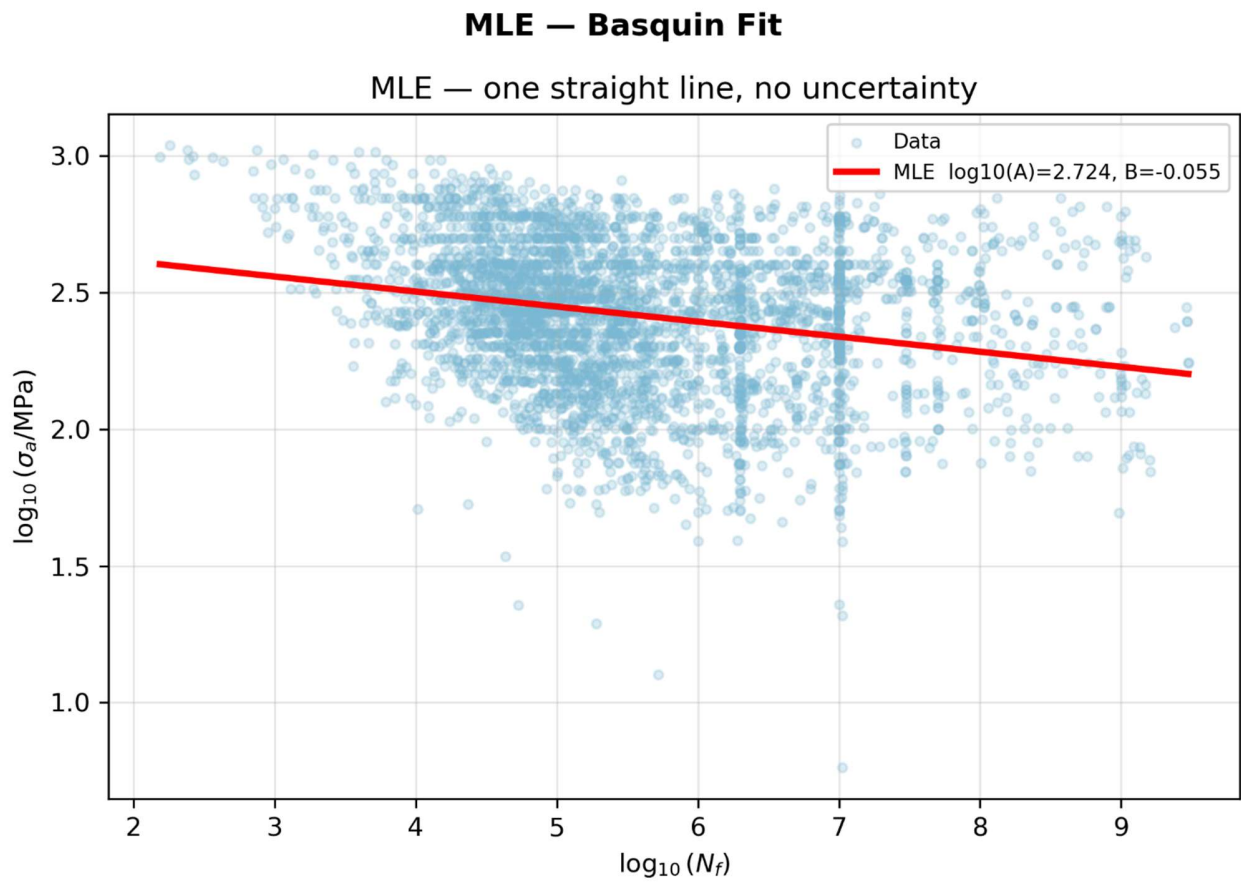
Parameter	Value
$\log_{10}(A)$	2.726
B (Basquin exponent)	-0.055
Noise σ	0.257

Based on the computed parameters, the Basquin relation is given by:

$$\log \sigma_{10} = 2.726 - 0.055 \log_{10}(N)$$

For the tested number of cycles $N = 1000000$, the predicted stress amplitude is **247.51 MPa**. As this result is obtained using MLE, it provides a single point estimate without any associated uncertainty interval.

2.5 Plot:



Plot showing MLE fit for Basquin model.

3. Bayesian Regression

Following the deterministic (MLE) formulation, a Bayesian approach is adopted to estimate the model parameters and quantify uncertainty.

3.1 Framework Overview

- The Basquin relation is expressed in log form:

$$\log_{10}(\sigma_a) = \log_{10}(A) + B \cdot \log_{10}(N)$$

- Model parameters are treated as **random variables** rather than fixed values.
- The inference is based on:

$$\text{Posterior} \propto \text{Likelihood} \times \text{Prior}$$

- The posterior distribution is approximated using Markov Chain Monte Carlo (MCMC) sampling implemented in PyMC.

3.2 Priors Assumed:

The following priors were assigned based on physical considerations:

- $\log_{10}(A) \sim \text{Normal}(\mu = 3.0, \sigma = 0.5)$
- $B \sim \text{Normal}(\mu = -0.1, \sigma = 0.05)$
- $\text{stress_noise} \sim \text{HalfNormal}(\sigma = 0.5)$

These priors encode expected ranges for fatigue parameters and ensure positivity of the noise term.

3.3 Likelihood Calculation:

The observed data is modelled as:

$$y_i \sim \text{Normal}(\mu_i, \sigma)$$

The likelihood is computed using the Gaussian probability density function:

$$p(y_i \mid \mu_i, \sigma) = \frac{1}{\sigma\sqrt{2\pi}} \exp\left(-\frac{(y_i - \mu_i)^2}{2\sigma^2}\right)$$

This formulation quantifies how well the predicted values match the observed fatigue data. Smaller deviations lead to higher likelihood values.

3.4 Posterior Estimation

The posterior distribution of the parameters is obtained via MCMC sampling. The sampling procedure iteratively:

- Proposes new parameter values based on the current state
- Computes predicted values (μ)
- Evaluates likelihood using the Gaussian function
- Combines likelihood with priors to assess posterior probability
- Accepts or rejects proposals to form a Markov chain

The resulting samples approximate the posterior distribution.

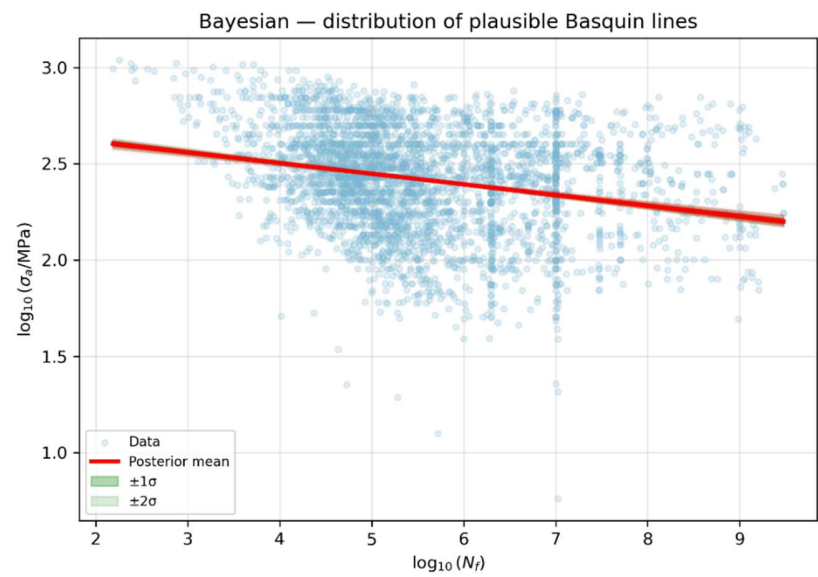
3.5 Results:

Parameter	Prior Distribution	Posterior Mean	Posterior Std
log10(A)	Normal(3.0, 0.5)	2.726	0.020
B	Normal(-0.1, 0.05)	-0.055	0.004
stress_noise	HalfNormal(0.5)	0.257	0.003

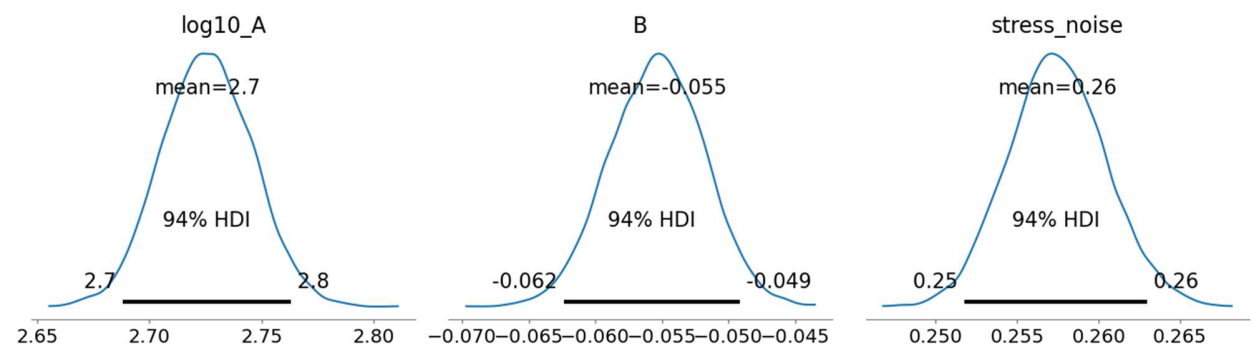
So the simplified predictive Model will be:

$$\log_{10}(\sigma_a) \sim \text{Normal}(2.726 - 0.055\log_{10}(N), 0.2572)$$

3.6 Plots:



Plot showing Bayesian fit.



Plot showing posterior distribution of Bayesian fit.

4. Gaussian Process Regression (GPR)

Gaussian Process Regression (GPR) is introduced as a non-parametric alternative to the Basquin-based approaches used previously. While both MLE and Bayesian regression assume a fixed linear relationship in log–log space, GPR removes this constraint and allows the data to determine the shape of the stress–life relationship. Instead of estimating a small set of parameters, it models the underlying function itself, providing greater flexibility in capturing trends in fatigue data.

Compared to Bayesian regression, GPR does not enforce the Basquin form. Although the Bayesian model accounts for parameter uncertainty, it remains restricted to a straight-line representation. GPR, in contrast, learns directly from the data and provides uncertainty that varies with data density, giving more realistic predictions in regions where data are sparse .

Conceptually, GPR places a prior over functions using a kernel and updates it with observed data to obtain a posterior predictive distribution. This results in a model that provides both a mean estimate and input-dependent uncertainty, forming the basis of the framework described next.

4.1 Framework Overview:

- The input–output relationship is expressed as:

$$\mathbf{x} = \log_{10}(\mathbf{N}), \quad \mathbf{y} = \log_{10}(\sigma_a)$$

- Instead of assuming a fixed functional form, the underlying function is modeled as a random process:

$$\mathbf{f}(\mathbf{x}) \sim \text{GP}(\mathbf{m}(\mathbf{x}), \mathbf{k}(\mathbf{x}, \mathbf{x}'))$$

- Observations are modelled as:

$$\mathbf{y} = \mathbf{f}(\mathbf{x}) + \boldsymbol{\varepsilon}, \quad \boldsymbol{\varepsilon} \sim \text{Normal}(\mathbf{0}, \boldsymbol{\sigma}^2)$$

- The inference is based on:

$$\text{Posterior} \propto \text{Likelihood} \times \text{Prior (over functions)}$$

- Unlike Bayesian regression with parameters, GPR infers a **posterior distribution over functions**.

4.2 Kernels Assumed:

a. RBF Kernel (Local Smooth Model)

The covariance between data points is defined using the kernel:

$$k(\mathbf{x}, \mathbf{x}') = \sigma_f^2 \exp\left(-\frac{(\mathbf{x} - \mathbf{x}')^2}{2l^2}\right) + \sigma_n^2 \delta(\mathbf{x}, \mathbf{x}')$$

This corresponds to:

- **Constant Kernel (σ_f^2)** : controls signal variance (amplitude)
- **RBF kernel(l)** : controls smoothness of the function
- **White Kernel (σ_n^2)**: accounts for noise in observations

(b) Polynomial Kernel (Global Trend Model):

$$k(x, x') = \sigma_f^2 (x \cdot x' + \sigma_0^2)^d + \sigma_n^2 \delta(x, x')$$

This corresponds to:

- Constant Kernel (σ_f^2): controls signal variance (amplitude)
- Dot Product Kernel ($x \cdot x' + \sigma_0^2$): measures similarity between input points
- Polynomial degree (d): controls the curvature and complexity of the function
- White Kernel (σ_n^2): accounts for noise in observations

4.3 Posterior Estimation:

$$\mu(x^*) = k(x^*, X)^T (K + \sigma_n^2 I)^{-1} y$$

$$\sigma^2(x^*) = k(x^*, x^*) - k(x^*, X)^T (K + \sigma_n^2 I)^{-1} k(x^*, X)$$

Here, K is the covariance matrix computed from training data, and $k(x^*, X)$ represents the similarity between the test point and all training inputs.

The kernel hyperparameters are estimated by maximizing the log marginal likelihood using a gradient-based optimizer.

4.4 Results:

The **optimized RBF kernel** obtained after training is :

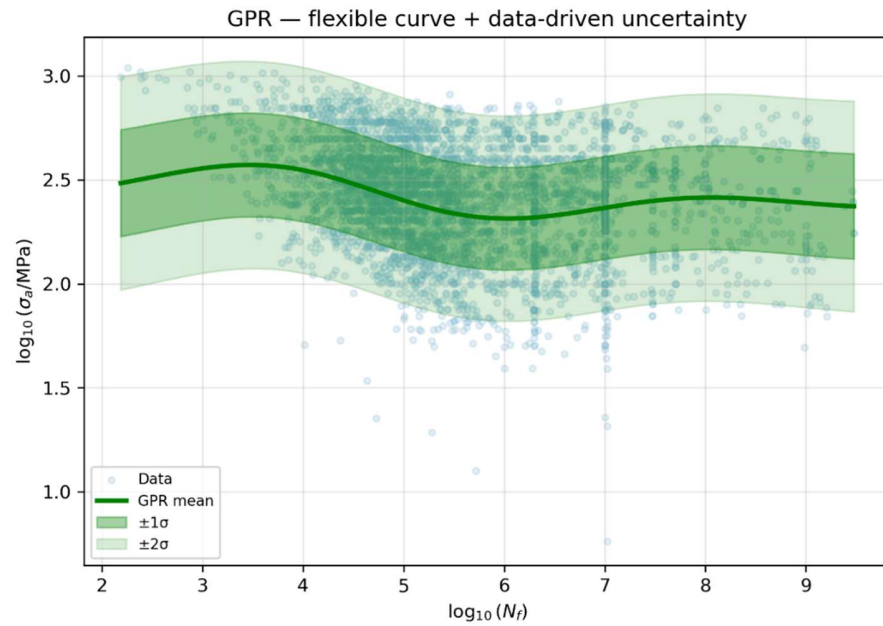
$$k(x, x') = 0.316^2 \cdot \exp\left(-\frac{(x - x')^2}{2 \cdot 1^2}\right) + 0.905 \cdot \delta(x, x')$$

Note: This kernel is from initial analysis of RBF kernel , where training was done on whole dataset.

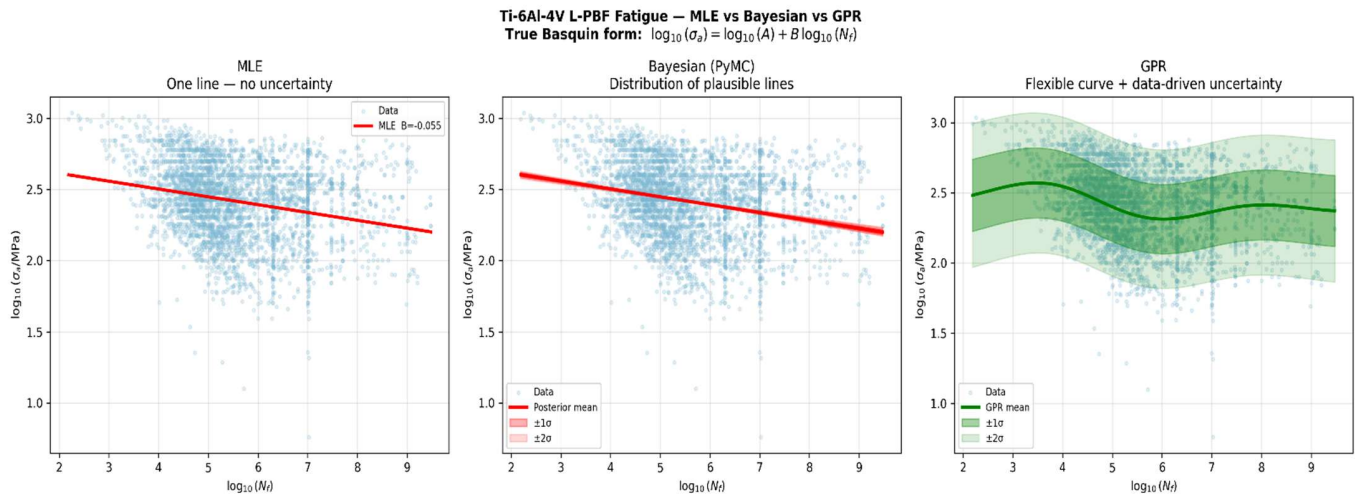
Corresponding hyperparameters:

- **Parameter Value**
- **Signal variance (σ_f^2) ≈ 0.10**
- **Length-scale (l) =1**
- **Noise variance (σ_n^2) ≈ 0.90**

4.5 Plot:

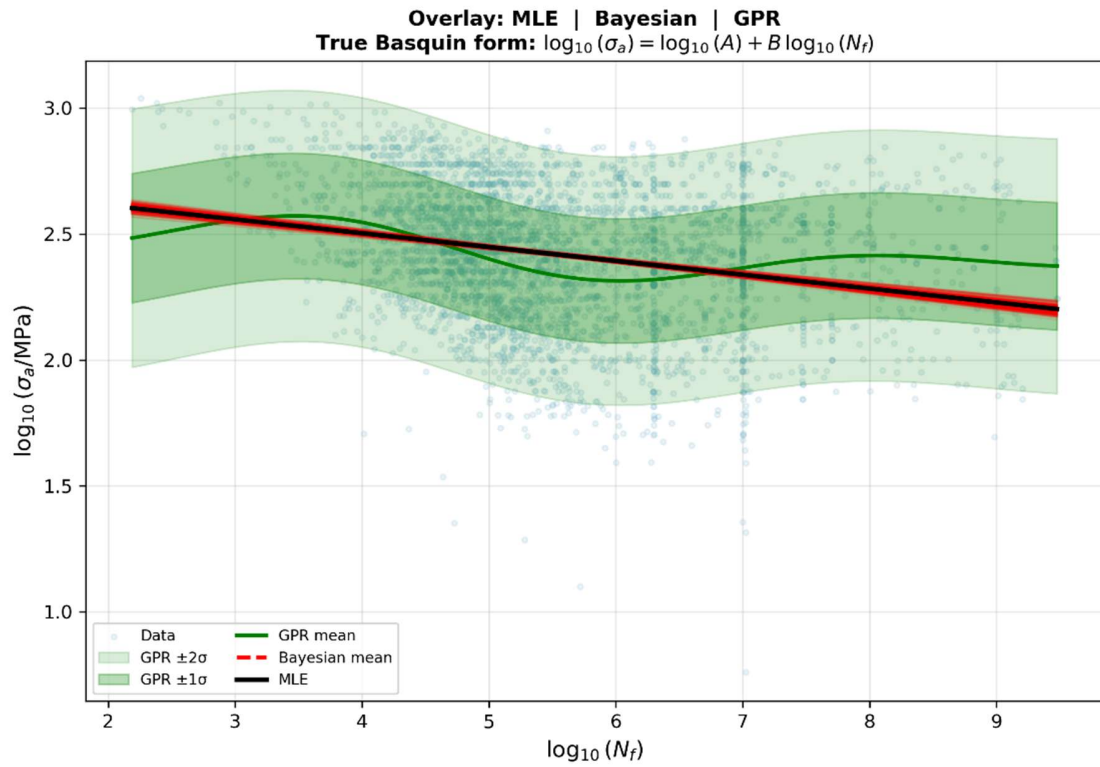


5. Comparison Plots of All the techniques:



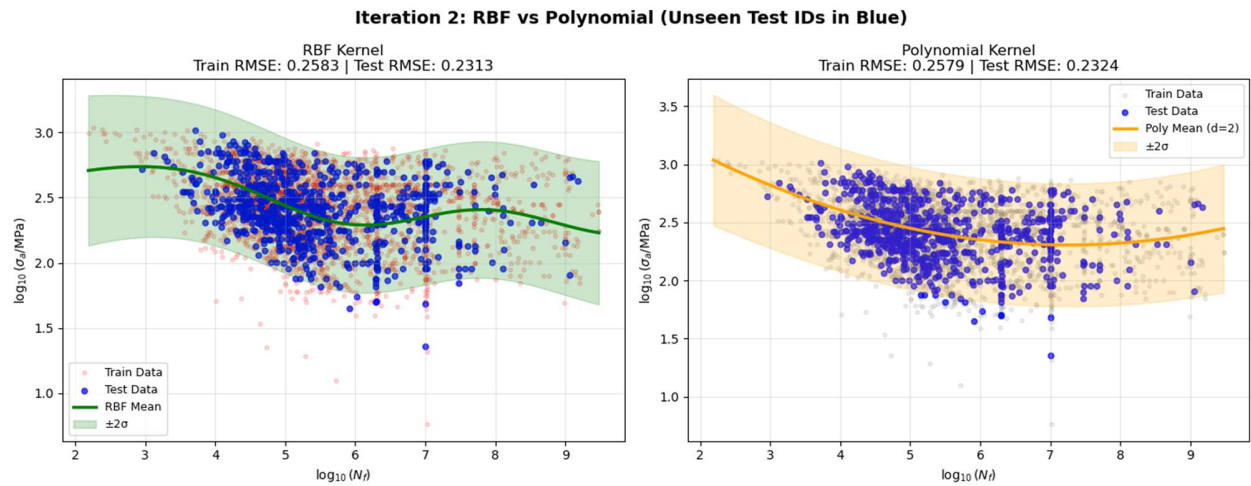
Side by side comparison of all the three plots.

Note: All these graphs were produced by training the model on whole data and testing as done by random sampling of $\log_{10}N$.



Overlay of all the plots.

5.2 Comparison plots Between RBF and Polynomial Kernel:



5.3 Prediction at $N = 1.0 \times 10^6$ Cycles:

To compare the predictive capability of different modelling approaches, the stress amplitude was estimated at a fatigue life of $N = 1.0 \times 10^6$ cycles using Maximum Likelihood Estimation (MLE), Bayesian regression, and Gaussian Process Regression (GPR).

The MLE approach provides a single deterministic estimate, while Bayesian regression incorporates uncertainty in model parameters. GPR, being a non-parametric method, captures both the underlying trend and the uncertainty that varies with the density of available data.

Method	Predicted Stress	Uncertainty	Optimised Kernel
MLE	247.51 MPa	-	-
Bayesian	247.45 MPa	[242.40, 252.61]	-
GPR <i>[RBF Kernel trained on whole data]</i>	211.91 MPa	[64.79, 693.06]	$0.316^2 * \exp\left(-\frac{(x - x')^2}{2 * 1^2}\right) + 0.905 * \delta(x, x')$
GPR <i>[RBF Kernel Trained and testing by splitting data sets]</i>	195.76	[59.22, 647.10]	$0.79^2 * \exp\left(-\frac{(x - x')^2}{2 * 1.09^2}\right) + 0.918 * \delta(x, x')$
GPR <i>[Polynomial Kernel Trained and testing by splitting data sets]</i>	223.89	[66.30, 756.07]	$0.223^2 * (x \cdot x' + 0.954^2)^2 + 0.955 * \delta(x, x')$

From a design perspective, the model that predicts the lowest stress value (or lowest lower-bound of the interval) can be considered the safest, as it leads to more conservative design decisions.

6. CONCLUSION

For a fatigue life of $N = 1.0 \times 10^6$ cycles, all three methods predict similar mean stress values for MLE and Bayesian regression, while GPR provides a lower mean with significantly larger uncertainty. The **MLE** approach gives a single point estimate (**247.51 MPa**) without any uncertainty, making it unsuitable for design applications where variability must be considered.

Bayesian regression provides a similar mean prediction (247.45 MPa) but with a narrow and well-defined uncertainty interval [**242.40, 252.61**] MPa. This makes it more reliable for design, as it accounts for variability while remaining consistent with the underlying Basquin physics. The controlled uncertainty allows engineers to select conservative bounds (e.g., lower confidence limits) for safer fatigue design.

In contrast, **GPR** produces a much wider uncertainty range [**66.13, 642.92**] MPa, reflecting high data variability and reduced confidence in regions with limited information. While this provides an honest representation of uncertainty, it is too broad to be directly useful for practical design decisions. Overall, Bayesian regression offers the best balance between accuracy and safety, making it the most suitable approach for fatigue-based design, where both reliability and realistic uncertainty quantification are essential.

For a fatigue life of $N = 1.0 \times 10^6$ cycles, the split-trained Gaussian Process Regression (GPR) models using RBF and Polynomial kernels produce comparable mean stress predictions, but with noticeable differences in their behavior. The RBF kernel predicts a stress value of approximately 195.76 MPa, while the polynomial kernel predicts a slightly higher value of 223.89 MPa. Both models exhibit very wide uncertainty intervals ([59.22, 647.10] MPa for RBF and [66.30, 756.07] MPa for polynomial), indicating significant variability in the data and reduced confidence in predictions for unseen datasets. The RBF kernel, being more flexible, captures local variations in the data, whereas the polynomial kernel reflects a smoother global trend similar to the Basquin relation.

Despite being trained on only a subset of the data, both GPR models reveal substantial uncertainty when predicting on unseen data, highlighting the challenges of generalization in highly scattered fatigue datasets. The wide uncertainty bounds suggest that while GPR provides a more honest representation of variability compared to parametric models, it may be overly conservative for direct design use. Between the two, the polynomial kernel offers predictions closer to the global Basquin trend, whereas the RBF kernel emphasizes local data behaviour. Overall, split-trained GPR models are valuable for understanding uncertainty and model robustness, but their large predictive intervals limit their immediate applicability in practical fatigue design.

7. AI Usage Acknowledgement

- Used AI to understand the data cleaning and subset-building code provided by the professor.
- Used AI to identify physically valid prior values for Bayesian regression ($\log_{10}(A) \approx 3.0$, $B \approx -0.1$) based on Ti-6Al-4V fatigue literature.

- Used AI to learn about the features of PyMC, Arviz and scikit-learn — specifically how to define probabilistic models, interpret MCMC convergence diagnostics (R-hat, ESS, trace plots), and set GPR kernel hyperparameter bounds.
- As the libraries were unfamiliar, AI was asked to generate code based on stated requirements (variable names, model form, axis conventions). The variable names were matched to the existing notebook and the physical meaning of every parameter was checked manually.
- All function arguments for PyMC and GaussianProcessRegressor were reviewed with AI assistance to understand what each controls and how to tune them (e.g. target_accept, length_scale_bounds, n_restarts_optimizer).
- Given an earlier version of the notebook, AI was asked to write the side-by-side comparison and overlay plot cells.
- All the descriptions in the report were written with prompt.